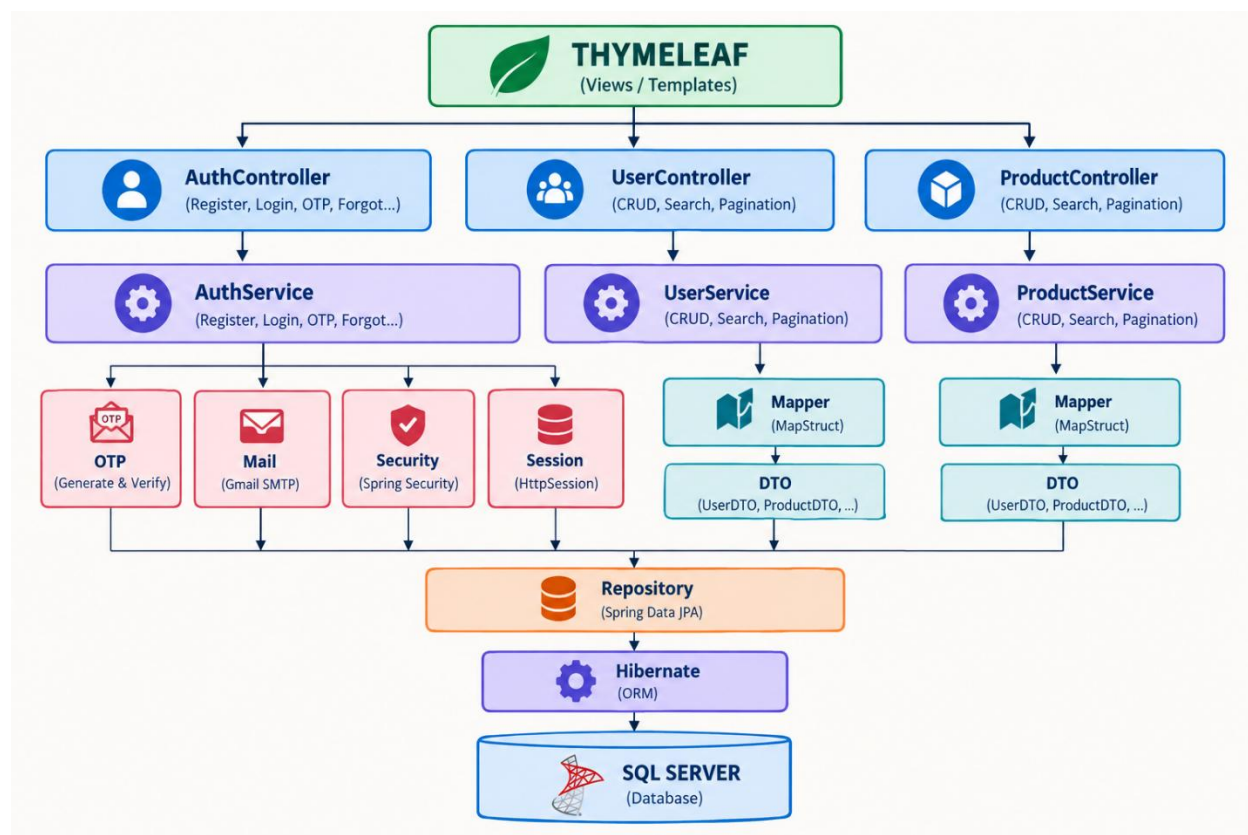


HƯỚNG DẪN SPRING BOOT + SECURITY

Bài tập:

Cho các bảng Users, Roles (user, admin), OtpToken, Products. Mỗi quan hệ 1 user - n product. Upload images vào cloud cloudanry. Xây dựng code chi tiết cho chức năng Register xác nhận OTP qua mail, Login lưu session, forgotpassword gửi OTP qua mail, CRUD bảng user, product, tìm kiếm phân trang cho bảng user, cho bảng product, đếm số user, product của user bằng spring boot 4.1.1 + security mới nhất + mapper DTO to Entity và ngược lại bằng mapstruct, các view bằng thymeleaf + sql server.



Chức năng:

Authentication

- Register
- Gửi OTP email
- Xác nhận OTP
- Resend OTP

- Login
- Spring Security Session
- Logout
- Forgot password
- Gửi OTP reset password
- Verify OTP
- Đổi password

User

1. CRUD User
2. Search User
3. Pagination User
4. Role USER/ADMIN
5. Đếm tổng User
6. Đếm Product của từng User

Product

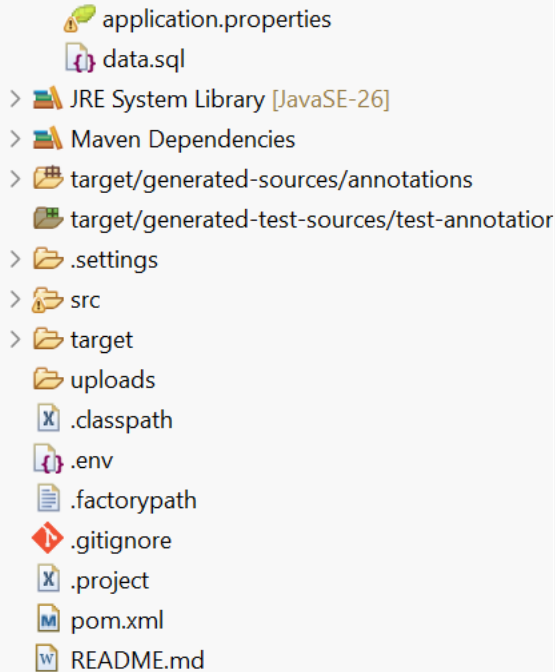
- CRUD Product
- Search Product
- Pagination Product
- Upload ảnh Cloudinary
- Product thuộc User
- Hiển thị số Product của User
- Xóa Product

. Công nghệ

Thành phần	Công nghệ
Backend	Spring Boot 4.1.1
Security	Spring Security 7.1.x
Java	JDK 26
Database	SQL Server
ORM	Spring Data JPA / Hibernate
View	Thymeleaf
Mapper	MapStruct 1.6.3
Email	Spring Mail
Image	Cloudinary
Validation	Jakarta Validation
Build	Maven
Authentication	Session
Password	BCrypt
Architecture	MVC + Service + Repository
Cấu trúc project:	

- shop-springboot-4-1-1 [boot]
 - src/main/java
 - vn.iotstar
 - config
 - CloudinaryConfig.java
 - EncodingConfig.java
 - SecurityConfig.java
 - controller
 - AuthController.java
 - ErrorController.java
 - HomeController.java
 - ProductController.java
 - UserController.java
 - dto
 - ForgotPasswordDTO.java
 - LoginDTO.java
 - ProductDTO.java
 - RegisterDTO.java
 - ResetPasswordDTO.java
 - UserDTO.java
 - VerifyOtpDTO.java
 - entity
 - OtpToken.java
 - Product.java
 - Role.java
 - User.java
 - mapper
 - ProductMapper.java
 - UserMapper.java
 - repository
 - OtpTokenRepository.java
 - ProductRepository.java
 - RoleRepository.java
 - UserRepository.java
 - security
 - CustomUserDetails.java
 - CustomUserDetailsService.java
 - service
 - impl

- impl
 - AuthServiceImpl.java
 - CloudinaryServiceImpl.java
 - EmailServiceImpl.java
 - OtpServiceImpl.java
 - ProductServiceImpl.java
 - UserServiceImpl.java
 - AuthService.java
 - CloudinaryService.java
 - CloudinaryUploadResult.java
 - EmailService.java
 - OtpService.java
 - ProductService.java
 - UserService.java
 - ShopApplication.java
- src/main/resources
 - static
 - css
 - app.css
 - templates
 - auth
 - forgot-password.html
 - login.html
 - register.html
 - reset-password.html
 - verify-otp.html
 - fragments
 - footer.html
 - header.html
 - layouts
 - layout.html
 - products
 - form.html
 - list.html
 - users
 - form.html
 - list.html
 - error.html
 - home.html



1. File pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>
    <relativePath />
  </parent>

  <groupId>vn.iotstar</groupId>
  <artifactId>shop-springboot-4-1-1</artifactId>
  <version>1.0.0</version>
  <name>shop-springboot-4-1-1</name>

  <properties>
    <java.version>26</java.version>
    <mapstruct.version>1.6.3</mapstruct.version>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <project.reporting.outputEncoding>UTF-
8</project.reporting.outputEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
```

```

        <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-mail</artifactId>
    </dependency>
    <dependency>
        <groupId>com.microsoft.sqlserver</groupId>
        <artifactId>mssql-jdbc</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.mapstruct</groupId>
        <artifactId>mapstruct</artifactId>
        <version>${mapstruct.version}</version>
    </dependency>
    <dependency>
        <groupId>org.mapstruct</groupId>
        <artifactId>mapstruct-processor</artifactId>
        <version>${mapstruct.version}</version>
        <scope>provided</scope>
    </dependency>
    <dependency>
        <groupId>com.cloudinary</groupId>
        <artifactId>cloudinary-http5</artifactId>
        <version>2.0.0</version>
    </dependency>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.thymeleaf.extras</groupId>
        <artifactId>thymeleaf-extras-springsecurity6</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>

```

```

        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.security</groupId>
        <artifactId>spring-security-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>nz.net.ultraq.thymeleaf</groupId>
        <artifactId>thymeleaf-layout-dialect</artifactId>
        <scope>compile</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <configuration>

                <encoding>UTF-8</encoding>

                <annotationProcessorPaths>
                    <path>
                        <groupId>org.mapstruct</groupId>
                        <artifactId>mapstruct-
processor</artifactId>
                        <version>${mapstruct.version}</version>
                    </path>
                    <path>
                        <groupId>org.projectlombok</groupId>
                        <artifactId>lombok</artifactId>
                    </path>
                </annotationProcessorPaths>
            </configuration>
        </plugin>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
</project>
=====

```

2. File application.properties

```

spring.config.import=optional:file:.env[.properties]
spring.application.name=shop

server.port=${SERVER_PORT:8080}

spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}

```

```

spring.jpa.hibernate.ddl-auto=${DDL_AUTO:update}
spring.jpa.show-sql=${SHOW_SQL:false}

spring.mail.host=${MAIL_HOST}
spring.mail.port=${MAIL_PORT}
spring.mail.username=${MAIL_USERNAME}
spring.mail.password=${MAIL_PASSWORD}

#jwt.secret=${JWT_SECRET}
#jwt.access-expiration=${JWT_ACCESS_EXPIRATION:900000}
#jwt.refresh-expiration=${JWT_REFRESH_EXPIRATION:604800000}

spring.datasource.driverClassName=com.microsoft.sqlserver.jdbc.SQLServerDriver

spring.jpa.properties.hibernate.format_sql=true

spring.thymeleaf.encoding=UTF-8
spring.thymeleaf.cache=false

spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true

cloudinary.cloud-name=${CLOUDINARY_CLOUD_NAME}
cloudinary.api-key=${CLOUDINARY_API_KEY}
cloudinary.api-secret=${CLOUDINARY_API_SECRET}

spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=20MB

spring.servlet.encoding.enabled=true
spring.servlet.encoding.charset=UTF-8
spring.servlet.encoding.force=true
spring.servlet.encoding.force-request=true
spring.servlet.encoding.force-response=true

spring.main.allow-bean-definition-overriding=true

```

3. Tạo biến môi trường .env trong project

```

# =====
# DATABASE
# =====
DB_URL=jdbc:sqlserver://localhost:1433;databaseName=webst3;encrypt=false;trustSer
verCertificate=true;sslProtocol=TLSv1.2;characterEncoding=UTF-8
DB_USERNAME=sa
DB_PASSWORD=fdfdfd

# =====
# JPA
# =====

```

```

DDL_AUTO=update
SHOW_SQL=true

# =====
# SERVER
# =====
SERVER_PORT=8080

# =====
# JWT
# =====
#JWT_SECRET=your-super-secret-key-change-this-in-production
#JWT_ACCESS_EXPIRATION=900000
#JWT_REFRESH_EXPIRATION=604800000

# =====
# SMTP
# =====
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=abc@gmail.com
MAIL_PASSWORD=zkhpxxfcalaknld

# =====
# CLOUDINARY
# =====
CLOUDINARY_CLOUD_NAME=dfdfdf
CLOUDINARY_API_KEY=576632571682623
CLOUDINARY_API_SECRET=ikPEbngxnKwAw-Xkvr1WVEaQZcI
=====

```

4. Tạo các Entity

```

package vn.iotstar.entity;

import jakarta.persistence.*;
import lombok.*;

import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "users",
      indexes = {
        @Index(name = "idx_users_username", columnList = "username"),
        @Index(name = "idx_users_email", columnList = "email")
      })
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String username;

    @Column(nullable = false, unique = true, length = 150)

```

```

    private String email;

    @Column(nullable = false)
    private String password;

    @Column(columnDefinition = "nvarchar(500)")
    private String fullName;

    @Builder.Default
    @Column(nullable = false)
    private boolean enabled = false;

    @ManyToOne(fetch = FetchType.EAGER, optional = false)
    @JoinColumn(name = "role_id", nullable = false)
    private Role role;

    @Builder.Default
    @OneToMany(mappedBy = "user", fetch = FetchType.LAZY)
    private List<Product> products = new ArrayList<>();
}
=====

package vn.iotstar.entity;

import jakarta.persistence.*;
import lombok.*;

import java.math.BigDecimal;
import java.time.LocalDateTime;

@Entity
@Table(name = "products",
        indexes = @Index(name = "idx_products_name", columnList = "name"))
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 2000, columnDefinition = "nvarchar(500)")
    private String name;

    @Column(length = 5000, columnDefinition = "nvarchar(500)")
    private String description;

    @Column(nullable = false, precision = 18, scale = 2)
    private BigDecimal price;

    @Column(length = 1000)
    private String imageUrl;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @Builder.Default

```

```

        @Column(nullable = false)
        private LocalDateTime createdAt = LocalDateTime.now();
    }
=====

package vn.iotstar.entity;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "roles")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Role {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = false, length = 30)
    private String name;
}
=====

package vn.iotstar.entity;

import jakarta.persistence.*;
import lombok.*;

import java.time.LocalDateTime;

@Entity
@Table(name = "otp_tokens",
        indexes = @Index(name = "idx_otp_email_type", columnList = "email,type"))
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class OtpToken {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 150)
    private String email;

    @Column(nullable = false, length = 100)
    private String otpHash;

    @Column(nullable = false, length = 30)
    private String type;

    @Column(nullable = false)
    private LocalDateTime expiresAt;

    @Column(nullable = false)
    private int attempts;

    @Builder.Default
    @Column(nullable = false)
    private boolean used = false;
}

```

```

@Column(nullable = false)
private LocalDateTime createdAt;
}

```

5. Tạo các DTO

```
package vn.iotstar.dto;
```

```
import jakarta.validation.constraints.*;
import lombok.Data;
```

```

@Data
public class UserDTO {
    private Long id;

    @NotBlank(message = "Username không được để trống")
    private String username;

    @NotBlank(message = "Email không được để trống")
    @Email(message = "Email không hợp lệ")
    private String email;

    @NotBlank(message = "Họ tên không được để trống")
    private String fullName;

    private boolean enabled;
    private String roleName;
    private long productCount;
}

```

```
=====
```

```
package vn.iotstar.dto;
```

```
import jakarta.validation.constraints.*;
import lombok.Data;
import org.springframework.web.multipart.MultipartFile;
```

```
import java.math.BigDecimal;
```

```

@Data
public class ProductDTO {
    private Long id;

    @NotBlank(message = "Tên sản phẩm không được để trống")
    private String name;

    private String description;

    @NotNull(message = "Giá không được để trống")
    @DecimalMin(value = "0.0", message = "Giá phải >= 0")
    private BigDecimal price;

    private String imageUrl;
    private Long userId;
    private String username;
}

```

```
    private MultipartFile image;
}
=====
```

```
package vn.iotstar.dto;
```

```
import jakarta.validation.constraints.*;
import lombok.Data;
```

```
@Data
public class RegisterDTO {
    @NotBlank(message = "Username không được để trống")
    private String username;

    @NotBlank(message = "Email không được để trống")
    @Email(message = "Email không hợp lệ")
    private String email;

    @NotBlank(message = "Mật khẩu không được để trống")
    @Size(min = 6, message = "Mật khẩu tối thiểu 6 ký tự")
    private String password;

    @NotBlank(message = "Xác nhận mật khẩu")
    private String confirmPassword;

    @NotBlank(message = "Họ tên không được để trống")
    private String fullName;
}
=====
```

```
package vn.iotstar.dto;
```

```
import jakarta.validation.constraints.NotBlank;
import lombok.Data;
```

```
@Data
public class LoginDTO {
    @NotBlank
    private String username;
    @NotBlank
    private String password;
}
=====
```

```
package vn.iotstar.dto;
```

```
import jakarta.validation.constraints.*;
import lombok.Data;
```

```
@Data
public class ResetPasswordDTO {
    @NotBlank @Email
    private String email;
}
```

```

    @NotBlank
    @Size(min = 6)
    private String password;

    @NotBlank
    private String confirmPassword;
}
=====

```

```
package vn.iotstar.dto;
```

```
import jakarta.validation.constraints.*;
import lombok.Data;
```

```

@Data
public class VerifyOtpDTO {
    @NotBlank @Email
    private String email;

    @NotBlank
    @Size(min = 6, max = 6)
    private String otp;
}
=====

```

```
package vn.iotstar.dto;
```

```
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import lombok.Data;
```

```

@Data
public class ForgotPasswordDTO {
    @NotBlank
    @Email
    private String email;
}
=====

```

6. Tạo các Mapper

```
package vn.iotstar.mapper;
```

```
import org.mapstruct.*;
import vn.iotstar.dto.UserDTO;
import vn.iotstar.entity.User;
```

```

@Mapper(componentModel = "spring",
        unmappedTargetPolicy = ReportingPolicy.IGNORE)
public interface UserMapper {
    @Mapping(target = "roleName", source = "role.name")
    UserDTO toDTO(User entity);
}

```

```

        @Mapping(target = "role", ignore = true)
        @Mapping(target = "products", ignore = true)
        @Mapping(target = "password", ignore = true)
        User toEntity(UserDTO dto);
    }
}

package vn.iotstar.mapper;

import org.mapstruct.*;
import vn.iotstar.dto.ProductDTO;
import vn.iotstar.entity.Product;

@Mapper(componentModel = "spring",
        unmappedTargetPolicy = ReportingPolicy.IGNORE)
public interface ProductMapper {
    @Mapping(target = "userId", source = "user.id")
    @Mapping(target = "username", source = "user.username")
    @Mapping(target = "image", ignore = true)
    ProductDTO toDTO(Product entity);

    @Mapping(target = "user", ignore = true)
    @Mapping(target = "createdAt", ignore = true)
    Product toEntity(ProductDTO dto);
}

=====

```

7. Tạo database

Khởi động SQL server lên và tạo 01 database (webst3)

8. Tạo các Repository

```

package vn.iotstar.repository;

import org.springframework.data.domain.*;
import org.springframework.data.jpa.repository.*;
import org.springframework.data.repository.query.Param;
import vn.iotstar.entity.User;
import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findByUsername(String username);

    Optional<User> findByEmail(String email);

    boolean existsByUsername(String username);

    boolean existsByEmail(String email);

    @Query("""
        select u from User u
        where lower(u.username) like lower(concat('%', :keyword, '%'))
        or lower(u.email) like lower(concat('%', :keyword, '%'))
        or lower(u.fullName) like lower(concat('%', :keyword, '%'))
    """)

```

```

    """
    Page<User> search(@Param("keyword") String keyword, Pageable pageable);

    @Query("select count(p) from Product p where p.user.id = :userId")
    long countProductsByUserId(@Param("userId") Long userId);

    @Query("""
        select u.id as id, count(p.id) as productCount
        from User u left join u.products p
        group by u.id
    """)
    java.util.List<Object[]> countProductsForUsers();
}

=====

package vn.iotstar.repository;

import org.springframework.data.domain.*;
import org.springframework.data.jpa.repository.*;
import org.springframework.data.repository.query.Param;
import vn.iotstar.entity.Product;

public interface ProductRepository extends JpaRepository<Product, Long> {

    @Query("""
        select p from Product p join fetch p.user u
        where lower(p.name) like lower(concat('%', :keyword, '%'))
        or lower(coalesce(p.description, '')) like lower(concat('%', :keyword,
'%'))
    """)
    Page<Product> search(@Param("keyword") String keyword, Pageable pageable);

    Page<Product> findByUserId(Long userId, Pageable pageable);

    long countByUserId(Long userId);
}

=====

package vn.iotstar.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import vn.iotstar.entity.Role;
import java.util.Optional;

public interface RoleRepository extends JpaRepository<Role, Long> {

    Optional<Role> findByName(String name);
}

=====

package vn.iotstar.repository;

```

```

import org.springframework.data.jpa.repository.JpaRepository;
import vn.iotstar.entity.OtpToken;
import java.util.Optional;

public interface OtpTokenRepository extends JpaRepository<OtpToken, Long> {

    Optional<OtpToken> findTopByEmailAndTypeAndUsedFalseOrderByCreatedAtDesc(
        String email, String type);

    void deleteByEmailAndType(String email, String type);

    Optional<OtpToken> findTopByEmailAndTypeOrderByCreatedAtDesc(
        String email, String type);
}

```

9. Tạo các interface services

```

package vn.iotstar.service;

import org.springframework.data.domain.Page;
import vn.iotstar.dto.UserDTO;

public interface UserService {
    Page<UserDTO> findAll(String keyword, int page, int size);
    UserDTO findById(Long id);
    UserDTO create(UserDTO dto);
    UserDTO update(Long id, UserDTO dto);
    void delete(Long id);
    long countUsers();
    long countProducts(Long userId);
}
=====

package vn.iotstar.service;

import org.springframework.data.domain.Page;
import org.springframework.web.multipart.MultipartFile;
import vn.iotstar.dto.ProductDTO;

public interface ProductService {
    Page<ProductDTO> findAll(String keyword, int page, int size);
    ProductDTO findById(Long id);
    ProductDTO create(ProductDTO dto, MultipartFile image);
    ProductDTO update(Long id, ProductDTO dto, MultipartFile image);
    void delete(Long id);
    long countProducts();
    long countByUser(Long userId);
}
=====

package vn.iotstar.service;

```

```

public interface OtpService {
    void sendRegisterOtp(String email);
    boolean verifyRegisterOtp(String email, String otp);
    void sendResetPasswordOtp(String email);
    boolean verifyResetPasswordOtp(String email, String otp);
}
=====

package vn.iotstar.service;

public interface EmailService {
    void sendOtp(String email, String otp, String subject);
}
=====

package vn.iotstar.service;

import org.springframework.web.multipart.MultipartFile;

public interface ClouinaryService {
    ClouinaryUploadResult upload(MultipartFile file);
    void delete(String publicId);
}
=====

package vn.iotstar.service;

public record ClouinaryUploadResult(String url, String publicId) {}
=====

package vn.iotstar.service;

import vn.iotstar.dto.RegisterDTO;

public interface AuthService {
    void register(RegisterDTO dto);
    boolean verifyRegister(String email, String otp);
    void forgotPassword(String email);
    boolean verifyResetOtp(String email, String otp);
    void resetPassword(String email, String password);
}

```

10. Tạo các Implement Services

```

package vn.iotstar.service.impl;

import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.*;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import vn.iotstar.dto.UserDTO;

```

```

import vn.iotstar.entity.Role;
import vn.iotstar.entity.User;
import vn.iotstar.mapper.UserMapper;
import vn.iotstar.repository.RoleRepository;
import vn.iotstar.repository.UserRepository;
import vn.iotstar.service.UserService;

@Service
@RequiredArgsConstructor
public class UserServiceImpl implements UserService {
    private final UserRepository userRepository;
    private final RoleRepository roleRepository;
    private final UserMapper mapper;
    private final PasswordEncoder passwordEncoder;

    @Override @Transactional(readOnly = true)
    public Page<UserDTO> findAll(String keyword, int page, int size) {
        Pageable pageable = PageRequest.of(
            Math.max(page, 0), Math.max(size, 1),
            Sort.by(Sort.Direction.DESC, "id")
        );
        return userRepository.search(keyword == null ? "" : keyword, pageable)
            .map(user -> {
                UserDTO dto = mapper.toDTO(user);
                dto.setProductCount(
                    userRepository.countProductsByUserId(user.getId())
                );
                return dto;
            });
    }

    @Override @Transactional(readOnly = true)
    public UserDTO findById(Long id) {
        User user = userRepository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("User không tồn tại"));
        UserDTO dto = mapper.toDTO(user);
        dto.setProductCount(userRepository.countProductsByUserId(id));
        return dto;
    }

    @Override @Transactional
    public UserDTO create(UserDTO dto) {
        if (userRepository.existsByUsername(dto.getUsername()))
            throw new IllegalArgumentException("Username đã tồn tại");
        if (userRepository.existsByEmail(dto.getEmail()))
            throw new IllegalArgumentException("Email đã tồn tại");

        User user = mapper.toEntity(dto);
        Role role = roleRepository.findByName(
            dto.getRoleName() == null || dto.getRoleName().isBlank()
                ? "ROLE_USER" : dto.getRoleName()
        ).orElseThrow(() -> new IllegalArgumentException("Role không tồn tại"));

        user.setRole(role);
    }

```

```

        user.setPassword(passwordEncoder.encode("123456"));
        user.setEnabled(dto.isEnabled());
        return mapper.toDTO(userRepository.save(user));
    }

    @Override @Transactional
    public UserDTO update(Long id, UserDTO dto) {
        User user = userRepository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("User không tồn
tại"));

        if (!user.getEmail().equals(dto.getEmail())
            && userRepository.existsByEmail(dto.getEmail()))
            throw new IllegalArgumentException("Email đã tồn tại");

        user.setUsername(dto.getUsername());
        user.setEmail(dto.getEmail());
        user.setFullName(dto.getFullName());
        user.setEnabled(dto.isEnabled());

        if (dto.getRoleName() != null && !dto.getRoleName().isBlank()) {
            Role role = roleRepository.findByName(dto.getRoleName())
                .orElseThrow(() -> new IllegalArgumentException("Role không tồn
tại"));
            user.setRole(role);
        }
        return mapper.toDTO(userRepository.save(user));
    }

    @Override @Transactional
    public void delete(Long id) {
        User user = userRepository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("User không tồn
tại"));
        userRepository.delete(user);
    }

    @Override @Transactional(readOnly = true)
    public long countUsers() { return userRepository.count(); }

    @Override @Transactional(readOnly = true)
    public long countProducts(Long userId) {
        return userRepository.countProductsByUserId(userId);
    }
}

```

=====

```

package vn.iotstar.service.impl;

import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.*;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.multipart.MultipartFile;
import vn.iotstar.dto.ProductDTO;

```

```

import vn.iotstar.entity.Product;
import vn.iotstar.entity.User;
import vn.iotstar.mapper.ProductMapper;
import vn.iotstar.repository.ProductRepository;
import vn.iotstar.repository.UserRepository;
import vn.iotstar.service.*;

@Service
@RequiredArgsConstructor
public class ProductServiceImpl implements ProductService {
    private final ProductRepository productRepository;
    private final UserRepository userRepository;
    private final ProductMapper mapper;
    private final CloudinaryService cloudinaryService;

    @Override @Transactional(readOnly = true)
    public Page<ProductDTO> findAll(String keyword, int page, int size) {
        Pageable pageable = PageRequest.of(
            Math.max(page, 0), Math.max(size, 1),
            Sort.by(Sort.Direction.DESC, "id")
        );
        return productRepository.search(keyword == null ? "" : keyword, pageable)
            .map(mapper::toDTO);
    }

    @Override @Transactional(readOnly = true)
    public ProductDTO findById(Long id) {
        return mapper.toDTO(productRepository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("Product không tồn
tại"))));
    }

    @Override @Transactional
    public ProductDTO create(ProductDTO dto, MultipartFile image) {
        User user = userRepository.findById(dto.getUserId())
            .orElseThrow(() -> new IllegalArgumentException("User không tồn tại"));
        Product product = mapper.toEntity(dto);
        product.setUser(user);
        if (image != null && !image.isEmpty()) {
            CloudinaryUploadResult r = cloudinaryService.upload(image);
            product.setImageUrl(r.url() + "|" + r.publicId());
        }
        return mapper.toDTO(productRepository.save(product));
    }

    @Override @Transactional
    public ProductDTO update(Long id, ProductDTO dto, MultipartFile image) {
        Product product = productRepository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("Product không tồn
tại"));
        product.setName(dto.getName());
        product.setDescription(dto.getDescription());
        product.setPrice(dto.getPrice());

        if (image != null && !image.isEmpty()) {

```

```

        String old = product.getImageUrl();
        if (old != null && old.contains("|")) {
            cloudinaryService.delete(old.substring(old.indexOf('|') + 1));
        }
        CloudinaryUploadResult r = cloudinaryService.upload(image);
        product.setImageUrl(r.url() + "|" + r.publicId());
    }
    return mapper.toDTO(productRepository.save(product));
}

@Override @Transactional
public void delete(Long id) {
    Product product = productRepository.findById(id)
        .orElseThrow(() -> new IllegalArgumentException("Product không tồn tại"));
    String image = product.getImageUrl();
    if (image != null && image.contains("|"))
        cloudinaryService.delete(image.substring(image.indexOf('|') + 1));
    productRepository.delete(product);
}

@Override @Transactional(readOnly = true)
public long countProducts() { return productRepository.count(); }

@Override @Transactional(readOnly = true)
public long countByUser(Long userId) {
    return productRepository.countByUserId(userId);
}
}
=====

```

```
package vn.iotstar.service.impl;
```

```

import lombok.RequiredArgsConstructor;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import vn.iotstar.entity.OtpToken;
import vn.iotstar.repository.OtpTokenRepository;
import vn.iotstar.service.EmailService;
import vn.iotstar.service.OtpService;

```

```

import java.security.SecureRandom;
import java.time.LocalDateTime;

```

```
@Service
```

```
@RequiredArgsConstructor
```

```

public class OtpServiceImpl implements OtpService {
    private static final int MAX_ATTEMPTS = 5;
    private static final int OTP_MINUTES = 5;
    private final OtpTokenRepository repository;
    private final PasswordEncoder passwordEncoder;
    private final EmailService emailService;
    private final SecureRandom random = new SecureRandom();
}

```

```

private String generateOtp() {
    return "%06d".formatted(random.nextInt(1_000_000));
}

private void send(String email, String type, String subject) {
    repository.deleteByEmailAndType(email, type);
    String otp = generateOtp();
    OtpToken token = OtpToken.builder()
        .email(email)
        .otpHash(passwordEncoder.encode(otp))
        .type(type)
        .expiresAt(LocalDate.now().plusMinutes(OTP_MINUTES))
        .attempts(0)
        .used(false)
        .createdAt(LocalDate.now())
        .build();
    repository.save(token);
    emailService.sendOtp(email, otp, subject);
}

@Override @Transactional
public void sendRegisterOtp(String email) {
    send(email, "REGISTER", "Shop - Xác nhận đăng ký tài khoản");
}

@Override @Transactional
public void sendResetPasswordOtp(String email) {
    send(email, "RESET_PASSWORD", "Shop - OTP đặt lại mật khẩu");
}

private boolean verify(String email, String otp, String type) {
    OtpToken token = repository
        .findTopByEmailAndTypeAndUsedFalseOrderByCreatedAtDesc(email, type)
        .orElse(null);
    if (token == null || token.getExpiresAt().isBefore(LocalDate.now())
        || token.getAttempts() >= MAX_ATTEMPTS) return false;

    token.setAttempts(token.getAttempts() + 1);
    if (!passwordEncoder.matches(otp, token.getOtpHash())) {
        repository.save(token);
        return false;
    }
    token.setUsed(true);
    repository.save(token);
    return true;
}

@Override @Transactional
public boolean verifyRegisterOtp(String email, String otp) {
    return verify(email, otp, "REGISTER");
}

@Override @Transactional
public boolean verifyResetPasswordOtp(String email, String otp) {
    return verify(email, otp, "RESET_PASSWORD");
}

```

```

    }
}
=====

package vn.iotstar.service.impl;

import lombok.RequiredArgsConstructor;
import org.springframework.mail.SimpleMailMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.stereotype.Service;
import vn.iotstar.service.EmailService;

@Service
@RequiredArgsConstructor
public class EmailServiceImpl implements EmailService {
    private final JavaMailSender mailSender;

    @Override
    public void sendOtp(String email, String otp, String subject) {
        SimpleMailMessage message = new SimpleMailMessage();
        message.setTo(email);
        message.setSubject(subject);
        message.setText("""
            Xin chào,

            Mã OTP của bạn là: %s

            OTP có hiệu lực trong 5 phút và chỉ sử dụng một lần.
            Không chia sẻ mã này cho người khác.
            """.formatted(otp));
        mailSender.send(message);
    }
}
=====

```

```

package vn.iotstar.service.impl;

import com.cloudinary.Cloudinary;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;
import vn.iotstar.service.CloudinaryService;
import vn.iotstar.service.CloudinaryUploadResult;

import java.util.Map;

@Service
@RequiredArgsConstructor
public class CloudinaryServiceImpl implements CloudinaryService {
    private final Cloudinary cloudinary;

    @Override
    public CloudinaryUploadResult upload(MultipartFile file) {
        if (file == null || file.isEmpty())
            throw new IllegalArgumentException("Chưa chọn ảnh");
    }
}

```

```

String type = file.getContentType();
if (type == null || !type.startsWith("image/"))
    throw new IllegalArgumentException("Chỉ cho phép file hình ảnh");

try {
    Map<?, ?> result = cloudinary.uploader().upload(
        file.getBytes(),
        Map.of("folder", "shop/products")
    );
    return new CloudinaryUploadResult(
        String.valueOf(result.get("secure_url")),
        String.valueOf(result.get("public_id"))
    );
} catch (Exception e) {
    throw new IllegalStateException("Upload Cloudinary thất bại", e);
}
}

@Override
public void delete(String publicId) {
    if (publicId == null || publicId.isBlank()) return;
    try {
        cloudinary.uploader().destroy(
            publicId, Map.of("resource_type", "image")
        );
    } catch (Exception e) {
        throw new IllegalStateException("Xóa ảnh Cloudinary thất bại", e);
    }
}
}

```

=====

```

package vn.iotstar.service.impl;

import lombok.RequiredArgsConstructor;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import vn.iotstar.dto.RegisterDTO;
import vn.iotstar.entity.Role;
import vn.iotstar.entity.User;
import vn.iotstar.repository.RoleRepository;
import vn.iotstar.repository.UserRepository;
import vn.iotstar.service.AuthService;
import vn.iotstar.service.OtpService;

@Service
@RequiredArgsConstructor
public class AuthServiceImpl implements AuthService {
    private final UserRepository userRepository;
    private final RoleRepository roleRepository;
    private final PasswordEncoder passwordEncoder;
    private final OtpService otpService;
}

```

```

@Override @Transactional
public void register(RegisterDTO dto) {
    if (userRepository.existsByUsername(dto.getUsername()))
        throw new IllegalArgumentException("Username đã tồn tại");
    if (userRepository.existsByEmail(dto.getEmail()))
        throw new IllegalArgumentException("Email đã tồn tại");
    if (!dto.getPassword().equals(dto.getConfirmPassword()))
        throw new IllegalArgumentException("Mật khẩu xác nhận không đúng");

    Role role = roleRepository.findByName("ROLE_USER")
        .orElseThrow(() -> new IllegalStateException("Chưa có ROLE_USER"));

    User user = User.builder()
        .username(dto.getUsername())
        .email(dto.getEmail())
        .password(passwordEncoder.encode(dto.getPassword()))
        .fullName(dto.getFullName())
        .enabled(false)
        .role(role)
        .build();

    userRepository.save(user);
    otpService.sendRegisterOtp(dto.getEmail());
}

@Override @Transactional
public boolean verifyRegister(String email, String otp) {
    boolean ok = otpService.verifyRegisterOtp(email, otp);
    if (!ok) return false;
    User user = userRepository.findByEmail(email)
        .orElseThrow(() -> new IllegalArgumentException("User không tồn tại"));
    user.setEnabled(true);
    return true;
}

@Override @Transactional
public void forgotPassword(String email) {
    if (!userRepository.existsByEmail(email))
        throw new IllegalArgumentException("Email không tồn tại");
    otpService.sendResetPasswordOtp(email);
}

@Override
public boolean verifyResetOtp(String email, String otp) {
    return otpService.verifyResetPasswordOtp(email, otp);
}

@Override @Transactional
public void resetPassword(String email, String password) {
    User user = userRepository.findByEmail(email)
        .orElseThrow(() -> new IllegalArgumentException("Email không tồn tại"));
    user.setPassword(passwordEncoder.encode(password));
}
}

```

11. Tạo các Configs

```

package vn.iotstar.config;

import com.cloudinary.Cloudinary;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.*;

import java.util.Map;

@Configuration
public class CloudinaryConfig {
    @Bean
    Cloudinary cloudinary(
        @Value("${cloudinary.cloud-name}") String cloudName,
        @Value("${cloudinary.api-key}") String apiKey,
        @Value("${cloudinary.api-secret}") String apiSecret) {
        return new Cloudinary(Map.of(
            "cloud_name", cloudName,
            "api_key", apiKey,
            "api_secret", apiSecret
        ));
    }
}

=====
package vn.iotstar.config;

import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.*;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import vn.iotstar.security.CustomUserDetailsService;

@Configuration
@RequiredArgsConstructor
public class SecurityConfig {
    private final CustomUserDetailsService userDetailsService;

    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .userDetailsService(userDetailsService)
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(
                    "/", "/login", "/register", "/verify-otp",
                    "/forgot-password", "/reset-password",
                    "/resend-register-otp", "/css/**", "/js/**"
                ).permitAll()
                .requestMatchers("/users/**").hasRole("ADMIN")
                .requestMatchers("/products/**").authenticated()
            );
    }
}

```

```

        .anyRequest().authenticated()
    )
    .formLogin(form -> form
        .loginPage("/login")
        .loginProcessingUrl("/login")
        .defaultSuccessUrl("/", true)
        .failureUrl("/login?error=true")
        .permitAll()
    )
    .logout(logout -> logout
        .logoutUrl("/logout")
        .logoutSuccessUrl("/login?logout=true")
        .invalidateHttpSession(true)
        .deleteCookies("JSESSIONID")
    )
    .sessionManagement(session -> session
        .maximumSessions(1)
        .maxSessionsPreventsLogin(false)
    );
    return http.build();
}
}
}

package vn.iotstar.config;

import org.springframework.boot.web.servlet.FilterRegistrationBean;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.filter.CharacterEncodingFilter;

@Configuration
public class EncodingConfig {
    @Bean
    FilterRegistrationBean<CharacterEncodingFilter> characterEncodingFilter() {
        CharacterEncodingFilter filter = new CharacterEncodingFilter();
        filter.setEncoding("UTF-8");
        filter.setForceEncoding(true);
        FilterRegistrationBean<CharacterEncodingFilter> registration = new
FilterRegistrationBean<>(filter);
        registration.setOrder(Integer.MIN_VALUE);
        return registration;
    }
}
}

```

12. Tạo các Controllers

```

package vn.iotstar.controller;

import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;
import vn.iotstar.dto.UserDTO;

```

```

import vn.iotstar.service.UserService;

@Controller
@RequestMapping("/users")
@RequiredArgsConstructor
public class UserController {
    private final UserService userService;

    @GetMapping
    public String list(@RequestParam(defaultValue = "") String keyword,
                      @RequestParam(defaultValue = "0") int page,
                      @RequestParam(defaultValue = "10") int size,
                      Model model) {
        model.addAttribute("users", userService.findAll(keyword, page, size));
        model.addAttribute("keyword", keyword);
        model.addAttribute("size", size);
        return "users/list";
    }

    @GetMapping("/create")
    public String create(Model model) {
        UserDTO dto = new UserDTO();
        dto.setEnabled(true);
        dto.setRoleName("ROLE_USER");
        model.addAttribute("userDTO", dto);
        model.addAttribute("mode", "create");
        return "users/form";
    }

    @PostMapping("/create")
    public String create(@Valid @ModelAttribute UserDTO dto,
                        BindingResult result,
                        Model model,
                        RedirectAttributes redirect) {
        if (result.hasErrors()) {
            model.addAttribute("mode", "create");
            return "users/form";
        }
        try {
            userService.create(dto);
            redirect.addFlashAttribute("success", "Tạo user thành công. Mật khẩu mặc định: 123456");
            return "redirect:/users";
        } catch (IllegalArgumentException e) {
            result.reject("user.error", e.getMessage());
            model.addAttribute("mode", "create");
            return "users/form";
        }
    }

    @GetMapping("/edit/{id}")
    public String edit(@PathVariable Long id, Model model) {
        model.addAttribute("userDTO", userService.findById(id));
        model.addAttribute("mode", "edit");
        return "users/form";
    }
}

```

```

    }

    @PostMapping("/edit/{id}")
    public String edit(@PathVariable Long id,
                      @Valid @ModelAttribute UserDTO dto,
                      BindingResult result,
                      Model model,
                      RedirectAttributes redirect) {
        if (result.hasErrors()) {
            model.addAttribute("mode", "edit");
            return "users/form";
        }
        try {
            userService.update(id, dto);
            redirect.addFlashAttribute("success", "Cập nhật user thành công.");
            return "redirect:/users";
        } catch (IllegalArgumentException e) {
            result.reject("user.error", e.getMessage());
            model.addAttribute("mode", "edit");
            return "users/form";
        }
    }

    @PostMapping("/delete/{id}")
    public String delete(@PathVariable Long id, RedirectAttributes redirect) {
        userService.delete(id);
        redirect.addFlashAttribute("success", "Xóa user thành công.");
        return "redirect:/users";
    }
}

=====

```

```

package vn.iotstar.controller;

import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;
import vn.iotstar.dto.ProductDTO;
import vn.iotstar.security.CustomUserDetails;
import vn.iotstar.service.ProductService;

@Controller
@RequestMapping("/products")
@RequiredArgsConstructor
public class ProductController {
    private final ProductService productService;

    @GetMapping
    public String list(@RequestParam(defaultValue = "") String keyword,

```

```

        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "10") int size,
        Model model) {
    model.addAttribute("products", productService.findAll(keyword, page, size));
    model.addAttribute("keyword", keyword);
    model.addAttribute("size", size);
    return "products/list";
}

@GetMapping("/create")
public String create(Model model) {
    model.addAttribute("productDTO", new ProductDTO());
    model.addAttribute("mode", "create");
    return "products/form";
}

@PostMapping("/create")
public String create(@Valid @ModelAttribute ProductDTO dto,
                    BindingResult result,
                    @RequestParam(required = false) MultipartFile image,
                    Authentication authentication,
                    Model model,
                    RedirectAttributes redirect) {
    if (result.hasErrors()) {
        model.addAttribute("mode", "create");
        return "products/form";
    }
    CustomUserDetails user = (CustomUserDetails) authentication.getPrincipal();
    dto.setUserId(user.getId());
    try {
        productService.create(dto, image);
        redirect.addFlashAttribute("success", "Tạo sản phẩm thành công.");
        return "redirect:/products";
    } catch (IllegalArgumentException e) {
        result.reject("product.error", e.getMessage());
        model.addAttribute("mode", "create");
        return "products/form";
    }
}

@GetMapping("/edit/{id}")
public String edit(@PathVariable Long id, Model model) {
    model.addAttribute("productDTO", productService.findById(id));
    model.addAttribute("mode", "edit");
    return "products/form";
}

@PostMapping("/edit/{id}")
public String edit(@PathVariable Long id,
                  @Valid @ModelAttribute ProductDTO dto,
                  BindingResult result,
                  @RequestParam(required = false) MultipartFile image,
                  Model model,
                  RedirectAttributes redirect) {
    if (result.hasErrors()) {

```

```

        model.addAttribute("mode", "edit");
        return "products/form";
    }
    //System.out.println("NAME = " + dto.getName());
    // System.out.println("DESCRIPTION = " + dto.getDescription());
    productService.update(id, dto, image);
    redirect.addFlashAttribute("success", "Cập nhật sản phẩm thành công.");
    return "redirect:/products";
}

@PostMapping("/delete/{id}")
public String delete(@PathVariable Long id, RedirectAttributes redirect) {
    productService.delete(id);
    redirect.addFlashAttribute("success", "Xóa sản phẩm thành công.");
    return "redirect:/products";
}
}

```

=====

```

package vn.iotstar.controller;

import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import vn.iotstar.service.ProductService;
import vn.iotstar.service.UserService;

@Controller
@RequiredArgsConstructor
public class HomeController {
    private final UserService userService;
    private final ProductService productService;

    @GetMapping("/")
    public String home(Model model) {
        model.addAttribute("userCount", userService.countUsers());
        model.addAttribute("productCount", productService.countProducts());
        return "home";
    }
}

```

=====

```

package vn.iotstar.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
@RequestMapping("/error")
public class ErrorController{

    public String error(Model model) {
        model.addAttribute("message", "Đã xảy ra lỗi.");
    }
}

```

```

        return "error";
    }
}
=====

```

```

package vn.iotstar.controller;

import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;
import vn.iotstar.dto.*;
import vn.iotstar.service.AuthService;
import vn.iotstar.service.OtpService;
import vn.iotstar.dto.RegisterDTO;

@Controller
@RequiredArgsConstructor
public class AuthController {
    private final AuthService authService;
    private final OtpService otpService;

    @GetMapping("/login")
    public String login() { return "auth/login"; }

    @GetMapping("/register")
    public String register(Model model) {
        model.addAttribute("registerDTO", new RegisterDTO());
        return "auth/register";
    }

    @PostMapping("/register")
    public String register(@Valid @ModelAttribute RegisterDTO dto,
                           BindingResult result,
                           RedirectAttributes redirect) {
        if (result.hasErrors()) return "auth/register";
        try {
            authService.register(dto);
            redirect.addFlashAttribute("success", "OTP đã được gửi đến email.");
            return "redirect:/verify-otp?email=" + dto.getEmail();
        } catch (IllegalArgumentException e) {
            result.reject("register.error", e.getMessage());
            return "auth/register";
        }
    }

    @GetMapping("/verify-otp")
    public String verifyPage(@RequestParam(required = false) String email, Model
model) {
        VerifyOtpDTO dto = new VerifyOtpDTO();
        dto.setEmail(email);
        model.addAttribute("verifyOtpDTO", dto);
    }
}

```

```

        return "auth/verify-otp";
    }

    @PostMapping("/verify-otp")
    public String verify(@Valid @ModelAttribute VerifyOtpDTO dto,
                        BindingResult result,
                        RedirectAttributes redirect) {
        if (result.hasErrors()) return "auth/verify-otp";
        if (!authService.verifyRegister(dto.getEmail(), dto.getOtp())) {
            result.reject("otp.error", "OTP không hợp lệ, hết hạn hoặc đã quá số lần
thử.");
            return "auth/verify-otp";
        }
        redirect.addFlashAttribute("success", "Xác nhận thành công. Hãy đăng nhập.");
        return "redirect:/login";
    }

    @PostMapping("/resend-register-otp")
    public String resend(@RequestParam String email, RedirectAttributes redirect) {
        otpService.sendRegisterOtp(email);
        redirect.addFlashAttribute("success", "Đã gửi lại OTP.");
        return "redirect:/verify-otp?email=" + email;
    }

    @GetMapping("/forgot-password")
    public String forgot(Model model) {
        model.addAttribute("forgotPasswordDTO", new ForgotPasswordDTO());
        return "auth/forgot-password";
    }

    @PostMapping("/forgot-password")
    public String forgot(@Valid @ModelAttribute ForgotPasswordDTO dto,
                        BindingResult result,
                        RedirectAttributes redirect) {
        if (result.hasErrors()) return "auth/forgot-password";
        try {
            authService.forgotPassword(dto.getEmail());
            redirect.addFlashAttribute("email", dto.getEmail());
            redirect.addFlashAttribute("success", "OTP đã được gửi.");
            return "redirect:/reset-password";
        } catch (IllegalArgumentException e) {
            result.reject("forgot.error", e.getMessage());
            return "auth/forgot-password";
        }
    }

    @GetMapping("/reset-password")
    public String reset(Model model) {
        ResetPasswordDTO dto = new ResetPasswordDTO();
        Object email = model.asMap().get("email");
        if (email != null) dto.setEmail(email.toString());
        model.addAttribute("resetPasswordDTO", dto);
        return "auth/reset-password";
    }

```

```

@PostMapping("/reset-password")
public String reset(@Valid @ModelAttribute ResetPasswordDTO dto,
                   BindingResult result,
                   @RequestParam String otp,
                   RedirectAttributes redirect) {
    if (!dto.getPassword().equals(dto.getConfirmPassword())) {
        result.reject("password.error", "Mật khẩu xác nhận không đúng.");
    }
    if (result.hasErrors()) return "auth/reset-password";

    if (!authService.verifyResetOtp(dto.getEmail(), otp)) {
        result.reject("otp.error", "OTP không hợp lệ hoặc đã hết hạn.");
        return "auth/reset-password";
    }

    authService.resetPassword(dto.getEmail(), dto.getPassword());
    redirect.addFlashAttribute("success", "Đổi mật khẩu thành công.");
    return "redirect:/login";
}
}

```

13. Tạo các security

```

package vn.iotstar.security;

import lombok.Getter;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import vn.iotstar.entity.User;

import java.util.Collection;
import java.util.List;

@Getter
public class CustomUserDetails implements UserDetails {
    private static final long serialVersionUID = 1L;
    private final Long id;
    private final String username;
    private final String password;
    private final boolean enabled;
    private final Collection<? extends GrantedAuthority> authorities;

    public CustomUserDetails(User user) {
        this.id = user.getId();
        this.username = user.getUsername();
        this.password = user.getPassword();
        this.enabled = user.isEnabled();
        this.authorities = List.of(
            (GrantedAuthority) () -> user.getRole().getName()
        );
    }

    @Override public Collection<? extends GrantedAuthority> getAuthorities() {
        return authorities;
    }
    @Override public String getPassword() { return password; }

```

```

        @Override public String getUsername() { return username; }
        @Override public boolean isEnabled() { return enabled; }
    }
    =====
package vn.iotstar.security;

import lombok.RequiredArgsConstructor;
import org.springframework.security.core.userdetails.*;
import org.springframework.stereotype.Service;
import vn.iotstar.repository.UserRepository;

@Service
@RequiredArgsConstructor
public class CustomUserDetailsService implements UserDetailsService {
    private final UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) {
        return userRepository.findByUsername(username)
            .map(CustomUserDetails::new)
            .orElseThrow(() ->
                new UsernameNotFoundException("Username không tồn tại"));
    }
}

```

14. Tạo CSS trong thư mục static

```

* { box-sizing: border-box; }
body { margin:0; font-family:Arial,sans-serif; background:#f5f7fb; color:#1f2937; }
a { color:#2563eb; text-decoration:none; }
.header { min-height:64px; display:flex; align-items:center; gap:28px; padding:0 5%; background:#111827; color:white; }
.header a { color:white; }
.brand { font-weight:800; font-size:20px; }
.header nav { display:flex; gap:18px; flex:1; }
.account { display:flex; gap:12px; align-items:center; }
.container { max-width:1200px; margin:30px auto; padding:0 20px; min-height:calc(100vh - 150px); }
.footer { text-align:center; padding:25px; background:#111827; color:white; }
.cards { display:grid; grid-template-columns:repeat(2,minmax(200px,1fr)); gap:20px; }
.card,.auth-card,.form-card { background:white; border-radius:12px; padding:25px; box-shadow:0 5px 25px rgba(0,0,0,.07); }
.card strong { font-size:36px; }
.toolbar { display:flex; justify-content:space-between; align-items:center; margin:20px 0; gap:15px; }
.toolbar form { display:flex; gap:8px; flex:1; }
input,textarea,select { width:100%; padding:11px; margin:5px 0 12px; border:1px solid #d1d5db; border-radius:7px; }
.toolbar input { margin:0; max-width:600px; }
.button { display:inline-block; border:0; border-radius:7px; padding:10px 16px; background:#2563eb; color:white; cursor:pointer; }
.button.secondary { background:#6b7280; }
.button.full { width:100%; }

```

```

.link-button { background:none; border:0; color:white; cursor:pointer; }
table { width:100%; background:white; border-collapse:collapse; box-shadow:0 3px 18px rgba(0,0,0,.05); }
th,td { padding:12px; border-bottom:1px solid #e5e7eb; text-align:left; vertical-align:middle; }
th { background:#f3f4f6; }
.actions { display:flex; gap:10px; align-items:center; }
.actions form { margin:0; }
.danger-link { border:0; background:none; color:#dc2626; cursor:pointer; }
.pagination { display:flex; gap:5px; justify-content:center; margin:25px; }
.pagination a { padding:8px 12px; border:1px solid #ddd; border-radius:5px; background:white; }
.pagination a.active { background:#2563eb; color:white; }
.alert { padding:12px; border-radius:7px; margin:12px 0; }
.alert.success { background:#dcfce7; color:#166534; }
.alert.error { background:#fee2e2; color:#991b1b; }
.error-text { color:#dc2626; display:block; margin-top:-8px; margin-bottom:8px; }
.auth-page { min-height:100vh; display:grid; place-items:center; padding:30px; }
.auth-card { width:min(450px,100%); }
.auth-card h1 { margin-top:0; }
.form-card { max-width:700px; }
.thumb { width:70px; height:70px; object-fit:cover; border-radius:7px; }
.preview { width:180px; max-height:180px; object-fit:cover; border-radius:8px; margin-bottom:15px; }
.quick-links { display:flex; gap:12px; margin-top:25px; }
.note { padding:12px; background:#fff7ed; border-radius:7px; }
.mt { margin-top:12px; }
@media(max-width:700px) {
  .header { flex-wrap:wrap; padding:15px; }
  .header nav { order:3; width:100%; }
  .cards { grid-template-columns:1fr; }
  .toolbar { flex-direction:column; align-items:stretch; }
  table { display:block; overflow-x:auto; }
}

```

15. Tạo các views layouts

```

<!DOCTYPE html>
<html
  xmlns:th="http://www.thymeleaf.org">

<head th:fragment="head(title)">

  <meta charset="UTF-8">

  <meta name="viewport"
    content="width=device-width, initial-scale=1">

  <title th:text="${title ?: 'IOTSTAR SHOP'}">
    IOTSTAR SHOP
  </title>

  <link rel="stylesheet"
    th:href="@{/css/app.css}">

</head>

```

```

<body>

<div th:fragment="page(content)">

    <header th:replace=~{fragments/header :: header}>
    </header>

    <main class="container"
        th:replace=~{content}>
    </main>

    <footer th:replace=~{fragments/footer :: footer}>
    </footer>

</div>

</body>
</html>
=====

```

```

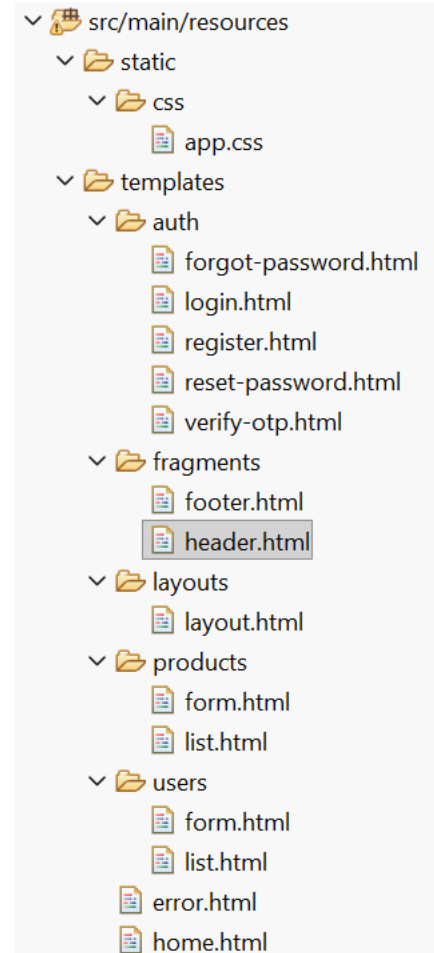
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>
<header th:fragment="header" class="header">
    <a th:href="@{/}" class="brand">IOTSTAR SHOP</a>
    <nav>
        <a th:href="@{/}">Dashboard</a>
        <a th:href="@{/products}">Products</a>
        <a th:if=~{#authorization.expression('hasRole('ADMIN')')}
            th:href="@{/users}">Users</a>
    </nav>
    <div class="account" th:if=~{#authorization.expression('isAuthenticated()')}>
        <span th:text=~{#authentication.name}></span>
        <form th:action="@{/logout}" method="post" style="display:inline">
            <button type="submit" class="Link-button">Logout</button>
        </form>
    </div>
    <div th:unless=~{#authorization.expression('isAuthenticated()')}>
        <a th:href="@{/login}">Login</a>
    </div>
</header>
</body>
</html>
=====

```

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>
<footer th:fragment="footer" class="footer">
    <span>© 2026 IOTSTAR SHOP - Spring Boot 4.1.1</span>
</footer>
</body>
</html>

```



16. Tạo các views contents

home.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace=~{layouts/layout :: head('Dashboard')}></head>
<body>
<div th:replace=~{layouts/layout :: page(~{::main})}>
<main>
    <h1>Dashboard</h1>
    <div th:if="${success}" class="alert success" th:text="${success}"></div>
    <div class="cards">
        <div class="card"><h3>Total Users</h3><strong
th:text="${userCount}">0</strong></div>
        <div class="card"><h3>Total Products</h3><strong
th:text="${productCount}">0</strong></div>
    </div>
    <div class="quick-links">
        <a class="button" th:href="@{/products}">Quản lý Products</a>
        <a class="button"
th:if="${#authorization.expression('hasRole(''ADMIN'')')}"
th:href="@{/users}">Quản lý Users</a>
    </div>
</main>
</div>
</body>
</html>
```

error.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace=~{layouts/layout :: head('Error')}></head>
<body>
    <div class="auth-page">
        <div class="auth-card">
            <h1>Có lỗi xảy ra</h1>
            <p th:text="${message}">Error</p>
            <a class="button" th:href="@{/}">Về trang chủ</a>
        </div>
    </div>
</body>
</html>
```

Auth/register.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace=~{layouts/layout :: head('Đăng ký')}></head>
<body>
    <div class="auth-page">
        <div class="auth-card">
            <h1>Đăng ký</h1>
```

```

        <form th:action="@{/register}" th:object="${registerDTO}"
            method="post">
            <div th:if="${#fields.hasGlobalErrors()}" class="alert
error">
                <p th:each="e : ${#fields.globalErrors()}"
th:text="${e}"></p>
            </div>
            <label>Username</label> <input th:field="*{username}"
                class="error-text" th:errors="*{username}"></small>
            <label>Email</label>
                <input type="email" th:field="*{email}"> <small
                class="error-text" th:errors="*{email}"></small>
            <label>Họ
                tên</label> <input th:field="*{fullName}"> <small
                class="error-text" th:errors="*{fullName}"></small>
            <label>Password</label>
                <input type="password" th:field="*{password}"> <small
                class="error-text" th:errors="*{password}"></small>
            <label>Confirm
                Password</label> <input type="password"
th:field="*{confirmPassword}">
                <small class="error-text"
th:errors="*{confirmPassword}"></small>
            <button class="button full" type="submit">Đăng ký & nhận
                OTP</button>
        </form>
        <p>
            <a th:href="@{/login}">Đã có tài khoản?</a>
        </p>
    </div>
</div>
</body>
</html>

```

Auth/login.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{layouts/layout :: head('Đăng nhập')}"></head>
<body>
    <div class="auth-page">
        <div class="auth-card">
            <h1>Đăng nhập</h1>
            <div th:if="${param.error}" class="alert error">Username/password
                không đúng hoặc tài khoản chưa được xác thực.</div>
            <div th:if="${param.logout}" class="alert success">Bạn đã đăng
                xuất.</div>
            <div th:if="${success}" class="alert success"
th:text="${success}"></div>
            <form th:action="@{/login}" method="post">
                <label>Username</label> <input name="username" required
autofocus>

```

```

        <label>Password</label> <input type="password"
name="password"
        required>
        <button class="button full" type="submit">Đăng
nhập</button>
    </form>
    <p>
        <a th:href="@{/register}">Tạo tài khoản</a>
    </p>
    <p>
        <a th:href="@{/forgot-password}">Quên mật khẩu?</a>
    </p>
</div>
</div>
</body>
</html>

```

Auth/reset-password.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{layouts/layout :: head('Đặt Lại mật khẩu')}"></head>
<body>
    <div class="auth-page">
        <div class="auth-card">
            <form th:action="@{/reset-password}"
th:object="${resetPasswordDTO}"
            method="post">
                <h1>Đặt lại mật khẩu</h1>
                <div th:if="${success}" class="alert success">
th:text="${success}"</div>
                <div th:if="${#fields.hasGlobalErrors()}" class="alert
error">
                    <p th:each="e : ${#fields.globalErrors()}">
th:text="${e}"</p>
                </div>
                <label>Email</label> <input type="email"
th:field="*{email}"
                required> <label>OTP</label> <input name="otp"
maxlength="6" required> <label>Mật khẩu mới</label>
<input
                type="password" th:field="*{password}" required>
<label>Xác
                nhận mật khẩu</label> <input type="password"
th:field="*{confirmPassword}"
                required>
                <button class="button full">Đổi mật khẩu</button>
            </form>
        </div>
    </div>
</body>
</html>

```

Auth/forgot-password.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{layouts/layout :: head('Quên mật khẩu')}"></head>
<body>
    <div class="auth-page">
        <div class="auth-card">
            <form th:action="@{/forgot-password}"
                  th:object="${forgotPasswordDTO}" method="post">
                <h1>Quên mật khẩu</h1>
                <div th:if="${#fields.hasGlobalErrors()}" class="alert
error">
                    <p th:each="e : ${#fields.globalErrors()}"
th:text="${e}"></p>
                </div>
                <label>Email</label> <input type="email"
th:field="*{email}"
                    required>
                <button class="button full">Gửi OTP</button>
            </form>
            <p>
                <a th:href="@{/Login}">Quay lại Login</a>
            </p>
        </div>
    </div>
</body>
</html>

```

Auth/verify-otp.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{layouts/layout :: head('Xác nhận OTP')}"></head>
<body>
    <div class="auth-page">
        <div class="auth-card">
            <form th:action="@{/verify-otp}" th:object="${verifyOtpDTO}"
                  method="post">
                <h1>Xác nhận OTP</h1>
                <div th:if="${success}" class="alert success"
th:text="${success}"></div>
                <div th:if="${#fields.hasGlobalErrors()}" class="alert
error">
                    <p th:each="e : ${#fields.globalErrors()}"
th:text="${e}"></p>
                </div>
                <label>Email</label> <input type="email"
th:field="*{email}"
                    required>
                <label>OTP 6 số</label> <input
th:field="*{otp}"
                    maxLength="6" inputmode="numeric" required>
                <button class="button full">Xác nhận</button>
            </form>
        </div>
    </div>
</body>
</html>

```

```

        <form th:action="@{/resend-register-otp}" method="post"
class="mt">
            <input type="hidden" name="email"
th:value="${verifyOtpDTO.email}">
            <button class="button secondary full">Gửi lại OTP</button>
        </form>
        <p>
            <a th:href="@{/login}">Quay lại Login</a>
        </p>
    </div>
</div>
</body>
</html>

```

User/list.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{layouts/layout :: head('Users')}"></head>
<body>
    <div th:replace="~{layouts/layout :: page(~{::main})}">
        <main>
            <h1>Quản lý Users</h1>
            <div th:if="${success}" class="alert success"
th:text="${success}"></div>
            <div class="toolbar">
                <form th:action="@{/users}" method="get">
                    <input name="keyword" th:value="${keyword}"
placeholder="Username, email, họ tên...">
                    <input
                        type="hidden" name="size" th:value="${size}">
                    <button class="button">Tìm kiếm</button>
                </form>
                <a class="button" th:href="@{/users/create}">+ Thêm
User</a>
            </div>
            <table>
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Username</th>
                        <th>Email</th>
                        <th>Họ tên</th>
                        <th>Role</th>
                        <th>Enabled</th>
                        <th>Products</th>
                        <th>Action</th>
                    </tr>
                </thead>
                <tbody>
                    <tr th:each="u : ${users.content}">
                        <td th:text="${u.id}"></td>
                        <td th:text="${u.username}"></td>
                        <td th:text="${u.email}"></td>
                        <td th:text="${u.fullName}"></td>

```

```

        <td th:text="${u.roleName}"></td>
        <td th:text="${u.enabled ? 'ACTIVE' :
'INACTIVE'}"></td>

        <td th:text="${u.productCount}"></td>
        <td class="actions"><a

            th:href="@{/users/edit/{id}(id=${u.id})}">Edit</a>
            <form
th:action="@{/users/delete/{id}(id=${u.id})}" method="post"
onsubmit="return confirm('Xóa
user này?')">
                <button class="danger-
link">Delete</button>
            </form></td>
        </tr>
        <tr th:if="${users.empty}">
            <td colspan="8">Không có dữ liệu.</td>
        </tr>
    </tbody>
</table>
<div class="pagination">
    <a th:if="${users.hasPrevious()}"
        th:href="@{/users(page=${users.number-
1},size=${size},keyword=${keyword})}"><<</a>
    <span
        th:each="i : ${#numbers.sequence(0, users.totalPages
> 0 ? users.totalPages - 1 : 0)}">
        <a th:classappend="${i == users.number ? 'active' :
''}"
            th:href="@{/users(page=${i},size=${size},keyword=${keyword})}"
            th:text="${i+1}"></a>
        </span> <a th:if="${users.hasNext()}"
            th:href="@{/users(page=${users.number+1},size=${size},keyword=${keyword})}">>>
    </div>
</main>
</div>
</body>
</html>

```

User/form.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{layouts/layout :: head(${mode == 'create' ? 'Thêm User' : 'Sửa
User'})}"></head>
<body>
<div th:replace="~{layouts/layout :: page(~{::main})}">
<main>

```

```

<form th:object="${userDTO}" method="post" enctype="multipart/form-data" accept-
charset="UTF-8" th:action="${mode == 'create'} ? @{/users/create} :
@{/users/edit/{id}(id=${userDTO.id})}">
    <h1 th:text="${mode == 'create'} ? 'Thêm User' : 'Sửa User'}"></h1>
    <label>Username</label>
    <input th:field="*{username}">
    <small class="error-text" th:errors="*{username}"></small>
    <label>Email</label>
    <input type="email" th:field="*{email}">
    <small class="error-text" th:errors="*{email}"></small>
    <label>Họ tên</label>
    <input th:field="*{fullName}">
    <small class="error-text" th:errors="*{fullName}"></small>
    <label>Role</label>
    <select th:field="*{roleName}">
        <option value="ROLE_USER">USER</option>
        <option value="ROLE_ADMIN">ADMIN</option>
    </select>
    <label><input type="checkbox" th:field="*{enabled}"> Enabled</label>
    <button class="button">Lưu</button>
    <a class="button secondary" th:href="@{/users}">Hủy</a>
</form>
<p th:if="${mode == 'create'}" class="note">Mật khẩu mặc định của user do Admin tạo:
<b>123456</b>.</p>
</main>
</div>
</body>
</html>

```

Products/list.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{layouts/layout :: head('Products')}">

</head>
<body>
    <div th:replace="~{layouts/layout :: page(~{::main})}">
        <main>
            <h1>Quản lý Products</h1>
            <div th:if="${success}" class="alert success"
th:text="${success}"></div>
            <div class="toolbar">
                <form th:action="@{/products}" method="get">
                    <input name="keyword" th:value="${keyword}"
placeholder="Tên sản phẩm, mô tả..."> <input
type="hidden"
                    name="size" th:value="${size}">
                    <button class="button">Tìm kiếm</button>
                </form>
                <a class="button" th:href="@{/products/create}">+ Thêm
Product</a>
            </div>
            <table>

```

```

<thead>
  <tr>
    <th>ID</th>
    <th>Ảnh</th>
    <th>Name</th>
    <th>Price</th>
    <th>User</th>
    <th>Action</th>
  </tr>
</thead>
<tbody>
  <tr th:each="p : ${products.content}">
    <td th:text="${p.id}"></td>
    <td> <span
th:unless="${p.imageUrl != null}">No
      image</span></td>
    <td th:text="${p.name}"></td>
    <td>
      th:text="${#numbers.formatDecimal(p.price, 0, 'COMMA', 2, 'POINT')}"></td>
      <td th:text="${p.username}"></td>
      <td class="actions"><a
        th:href="@{/products/edit/{id}(id=${p.id})}">Edit</a>
        <form
th:action="@{/products/delete/{id}(id=${p.id})}"
        method="post" onsubmit="return
confirm('Xóa product này?')">
        <button class="danger-
link">Delete</button>
        </form></td>
    </tr>
    <tr th:if="${products.empty}">
      <td colspan="6">Không có dữ liệu.</td>
    </tr>
  </tbody>
</table>
<div class="pagination">
  <a th:if="${products.hasPrevious()}"
    th:href="@{/products(page=${products.number-
1},size=${size},keyword=${keyword})}"><<</a>
  <span
    th:each="i : ${#numbers.sequence(0,
products.totalPages > 0 ? products.totalPages - 1 : 0)}">
    <a th:classappend="${i == products.number ? 'active'
: ''}"
    th:href="@{/products(page=${i},size=${size},keyword=${keyword})}"
    th:text="${i+1}"></a>
  </span> <a th:if="${products.hasNext()}"

```

```

        th:href="@{/products(page=${products.number+1},size=${size},keyword=${keyword})}">></a>
    </div>
</main>
</div>
</body>
</html>

```

Products/form.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head
    th:replace=~{layouts/layout :: head(
        ${mode == 'create' ? 'Thêm Product' : 'Sửa Product'})}
    >>
</head>
<body>

    <div th:replace=~{layouts/layout :: page(~{::main})}>

        <main>

            <h1
                th:text="${mode == 'create'
                    ? 'Thêm Product'
                    : 'Sửa Product'}">

            </h1>

            <form th:object="${productDTO}"
                th:with="actionUrl=${mode == 'create'
                    ? '/products/create'
                    : '/products/edit/' + productDTO.id}"
                th:action="${actionUrl}" method="post"
                enctype="multipart/form-data"
                accept-charset="UTF-8">

                <!-- ID -->
                <input type="hidden" th:field="*{id}">

                <!-- Tên sản phẩm -->
                <div>

                    <label for="name"> Tên sản phẩm </label> <input
id="name"
                    type="text" th:field="*{name}"> <small
class="error-text"
                    th:if="${#fields.hasErrors('name')}"
th:errors="*{name}">

                    </small>

```

```

    </div>

    <!-- Mô tả -->
    <div>

        <label for="description"> Mô tả </label>

        <textarea id="description" th:field="*{description}"
rows="5">
    </textarea>

    </div>

    <!-- Giá -->
    <div>

        <label for="price"> Giá </label> <input id="price"
type="number"
        step="0.01" th:field="*{price}"> <small
class="error-text"
        th:if="${#fields.hasErrors('price')}"
        th:errors="*{price}">
    </small>

    </div>

    <!-- Ảnh -->
    <div>

        <label for="image"> Ảnh </label> <input id="image"
type="file"
        name="image" accept="image/*">

    </div>

    <!-- Preview ảnh hiện tại -->
    <div th:if="${productDTO.imageUrl != null}">

    </div>

    <!-- Button -->
    <div>

        <button type="submit" class="button">Lưu</button>

```

```

        <a class="button secondary" th:href="@{/products}">
Hủy </a>

    </div>

</form>

</main>

</div>

</body>

</html>

```

17. Chạy project

Kích phải project và thực hiện start

Mở SQL server lên và tạo query để thêm Roles vào như sau:

```

INSERT INTO roles (name)
SELECT 'ROLE_USER'
WHERE NOT EXISTS (SELECT 1 FROM roles WHERE name = 'ROLE_USER');

```

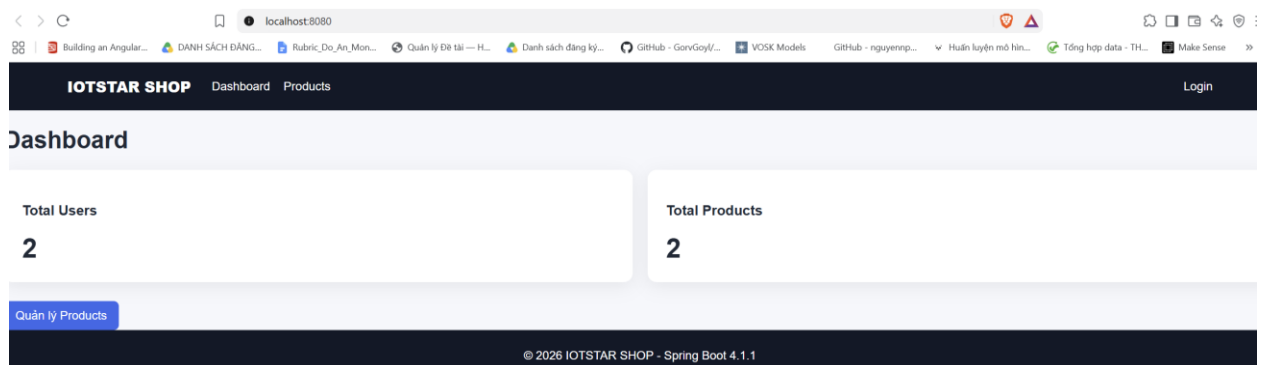
```

INSERT INTO roles (name)
SELECT 'ROLE_ADMIN'
WHERE NOT EXISTS (SELECT 1 FROM roles WHERE name = 'ROLE_ADMIN');

```

18. Kết quả

Kích đúp vào project trong cửa sổ Boot Dashboard để mở project và test kết quả như sau:



Đăng ký

Username

Email

Họ tên

Password

Confirm Password

Đăng ký & nhận OTP

[Đã có tài khoản?](#)

Đăng nhập

Username

Password

Đăng nhập

[Tạo tài khoản](#)

[Quên mật khẩu?](#)

Quên mật khẩu

Email

Gửi OTP

[Quay lại Login](#)

Sau khi đăng ký và kích hoạt OTP xong thì tiến hành đăng nhập.

localhost:8080

Building an Angular...

DANH SÁCH ĐĂNG...

Rubric_Do_An_Mon...

Quản lý Đề tài — H...

Danh sách đăng ký...

GitHub - GonyGoy/...

VOSK Models

GitHub - nguyennp...

Huấn luyện mô hìn...

Tổng hợp data - TH...

Make Sens

IOTSTAR SHOP

Dashboard

Products

trungnh

Logout

Dashboard

Total Users

2

Total Products

2

Quản lý Products

© 2026 IOTSTAR SHOP - Spring Boot 4.1.1

localhost:8080/products

Building an Angular...

DANH SÁCH ĐĂNG...

Rubric_Do_An_Mon...

Quản lý Đề tài — H...

Danh sách đăng ký...

GitHub - GonyGoy/...

VOSK Models

GitHub - nguyennp...

Huấn luyện mô hìn...

Tổng hợp data - TH...

Make Sense

IOTSTAR SHOP

Dashboard

Products

trungnh

Logout

Quản lý Products

Tên sản phẩm, mô tả...

Tim kiếm

+ Thêm Product

ID	Ảnh	Name	Price	User	Action
4		Điện thoại Oppo A95	656,565,656.00	trungnh	Edit Delete
3		Điện thoại Oppo A6	689,999.00	trungnh	Edit Delete

1



localhost:8080/products/create



Building an Angular...



DANH SÁCH ĐĂNG...



Rubric_Do_An_Mon...



Quản lý Đề tài -

IOTSTAR SHOP

Dashboard

Products

Thêm Product

Tên sản phẩm

Mô tả

Giá

Ảnh

Choose File

No file chosen

Lưu

Hủy

localhost:8080/products/edit/4

Building an Angular...

DANH SÁCH ĐĂNG...

Rubric_Do_An_Mon...

Quản lý Tài tài — I

IOTSTAR SHOP

Dashboard

Products

Sửa Product

Tên sản phẩm

Điện thoại Oppo A95

Mô tả

Điện thoại Oppo A95

Giá

656565656.00

Ảnh

Choose File

No file chosen



Lưu

Hủy

Khi đăng nhập với vai trò admin thì kết quả

localhost:8080

Building an Angular... DANH SÁCH ĐĂNG... Rubric_Do_An_Mon... Quản lý Dữ tài — H... Danh sách đăng ký... GitHub - GonyGoyl... VOSK Models GitHub - nguyennp... Huấn luyện mô hìn... Tổng hợp data - TH... Make Sense

IOTSTAR SHOPDashboardProductsUsersadminLogout

Dashboard

Total Users2

Total Products2

Quản lý Products

Quản lý Users

© 2026 IOTSTAR SHOP - Spring Boot 4.1.1

localhost:8080/users

Building an Angular... DANH SÁCH ĐĂNG... Rubric_Do_An_Mon... Quản lý Dữ tài — H... Danh sách đăng ký... GitHub - GonyGoyl... VOSK Models GitHub - nguyennp... Huấn luyện mô hìn... Tổng hợp data - TH... Make Sense

IOTSTAR SHOPDashboardProductsUsersadminLogout

Quản lý Users

Username, email, họ tên...

Tìm kiếm

+ Thêm User

ID	Username	Email	Họ tên	Role	Enabled	Products	Action
2	admin	trungnhspkt@gmail.com	Administrator	ROLE_ADMIN	ACTIVE	0	Edit Delete
1	trungnh	trungnh@hcmute.edu.vn	Nguyễn Hữu Trung	ROLE_USER	ACTIVE	2	Edit Delete

1

© 2026 IOTSTAR SHOP - Spring Boot 4.1.1